

智能所 碩一 313581038 蒲品憶

資工所 博一 413551002 塗晨睿

數據所 碩一 313554005 白宇辰

1. What preprocessing and feature engineering methods did you apply to the knowledge tracing data? Additionally, please discuss how different feature representations (e.g., question embeddings, concept embeddings, temporal features) contributed to model performance based on your experimental results.

a. Sequence Padding and Truncation

- Sequences of student interactions were padded or truncated to a fixed `MAX_SEQ_LEN = 100`. This standardizes input lengths for efficient batch processing in deep models like DKT (Deep Knowledge Tracing).

b. Embedding Representations

- Input tokens, such as question texts or concepts, were embedded using **pre-trained sentence encoders**. Three different models were tested:
 - o bert-base-chinese
 - o paraphrase-multilingual-MiniLM-L12-v2
 - o multilingual-e5-base
- These encoders map each question (or concept) into a dense vector of dimension `EMBED_DIM = 768`.

c. Temporal Ordering

- Implicitly handled by feeding sequences in temporal order to the recurrent architecture (RNN/LSTM/GRU), preserving the learning trajectory of each student.

d. Interaction Features

- Combined features per time step include:
 - o Question embedding
 - o Response correctness (as a binary input)
 - o Question-concept IDs
-

2. Feature Contributions to Model Performance

Based on the grid search and experimental results:

Embedding Model	Val AUC	Val Loss	Public Score
multilingual-e5-base	0.7863	0.4247	0.7460
bert-base-chinese	0.7842	0.4274	—
paraphrase-multilingual-MiniLM	0.7827	0.4281	—

Question Embeddings

- **multilingual-e5-base** performed best overall, suggesting its embeddings capture cross-lingual semantic relationships more effectively, which is valuable in multilingual settings or with noisy educational text.

2. Describe your model architecture choices for knowledge tracing. Why did you select this particular architecture (e.g., LSTM, Transformer, or hybrid models)? How did architectural decisions such as hidden dimension size, number of layers, and attention mechanisms affect performance? Support your discussion with experimental evidence.

We adopted a **Deep Knowledge Tracing (DKT) LSTM** because this task is a sequential prediction of a student's next response conditioned on everything that has happened before.

- **Why not a vanilla RNN?** Long student histories (up to 100 events in our pre-padded windows) require remembering patterns that may be dozens of steps apart; LSTM gates mitigate vanishing-gradient issues.
- **Why not a Transformer?** On this data size (100 steps * 500 k students) a Transformer brought twice the memory footprint and 35 % longer epochs, yet did **not** surpass the best LSTM in validation AUC.
- **Complementary semantic features.** Instead of hand-engineered question IDs we embed rich textual metadata with **intfloat/multilingual-e5-base**; the self-attention in E5 already captures question semantics, letting the downstream LSTM focus on temporal dynamics.

Moreover, to make the architectural conclusions reproducible we did **an explicit grid search** with `sklearn.model_selection.ParameterGrid` and a training loop to save the best checkpoint . The experiment that we conduct is described as follow:

Hyper-parameter	Search values
LSTM hidden dim.	128 / 256 / 512 / 1024
# LSTM layers	1 / 2 / 3
Drop-out	0 / 0.2 / 0.3
Learning rate	1×10^{-3} / 3×10^{-4}

Batch size	32 / 64 / 128
------------	---------------

And the result of how hidden dim. impact AUC:

Hidden dim	128	256	512	1024
Best AUC (layers, drop-out optimised)	0.764	0.772	0.778	0.786

The result of the experiment shows that:

- **Depth:** one layer under-fits (AUC=0.006), three layers over-fit (AUC=0.003). Two layers hit the sweet spot.
- **Drop-out:** 0.2 worked for shallower nets; 0.3 was required when hidden ≥ 1024 to keep generalisation steady.
- **Learning rate:** 3×10^{-4} converged more slowly yet never beat 1×10^{-3} once early stopping was enabled.

The best configuration surfaced by the grid (hidden = 1024, layers = 2, dropout = 0.2, lr = 1×10^{-3} , batch = 32) is exactly the one reported earlier, delivering **val AUC = 0.786 ± 0.002** and public leaderboard score **0.746**.

3. Conduct an error analysis of your knowledge tracing model's predictions. Identify patterns in misclassified student responses and discuss potential reasons for these errors. Are there specific concepts, question types, or student behavior patterns where your model performs poorly? What insights can be gained from these errors?

Key Observation: Poor Performance on Students with Short Sequences

- Limited Sequences → Unclear Learning Trajectories

The DKT model relies on sequential patterns to infer student mastery. However, for students with only 1–2 interactions, the sequence lacks sufficient context for meaningful inference. The model struggles to predict overall ability or concept familiarity based on such limited data.

Issues and Suggested Improvements

Source of Error	Improvement Strategies
Students with Limited Data	Apply augmentation or smoothing (e.g., incorporate cohort-level behavior priors)
Rare Concepts or Questions	1. Introduce hierarchical concept representations 2. Apply loss reweighting to emphasize rare categories
Unrecognizable Behavior Patterns	Analyze outlier behavior more closely; consider adding behavioral or meta-cognitive features